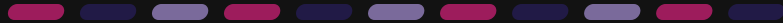


Variables

Photo of faculty



Variables

A core of programming

Variables hold *variable* value

In the context of *Computer Science* variables are a **name** with an associated **value**

Patterns with variables

We'll look at some common ways to use variables in Scratch

Each shows how variables hold values and change them over time

The goal is

- to develop an intuitive understanding of how variables work
- to understand which situations variables will be helpful in
- to recall common patterns that are useful for a large majority of programs

Pattern 1: Configuration Bundle

The problem: How do you make the game *feel* right?

Player movement uses multiple connected values:

- `speed` — how fast you move sideways
- `jumpForce` — how high you go
- `gravity` — how fast you come back down

These variables form a **system** — change one, and the whole feel of the game changes.

Pattern 1: Configuration Bundle

```

1  event _____
2  | when [flag v] clicked
3  | variables _____
4  | set [speed v] to [5]
5  | variables _____
6  | set [jumpForce v] to [12]
7  | variables _____
8  | set [gravity v] to [1]
9  |
10 |
11 | control _____
12 | | forever
13 | | | motion _____
14 | | | change x by [speed]
15 | | | change y by [jumpForce]
16 | | | change y by [-gravity]
17 | |
18 |

```

These variables are used **inside other blocks** — they control how the sprite moves.

Pattern 1: Configuration Bundle

Different preset combinations produce completely different game feel:

speed	jumpForce	gravity	Feel
3	8	0.5	Floaty, slow moon-jump
5	12	1	Standard platforming
8	15	2	Fast, heavy

The variables work as a **coordinated system** — real game developers tune these together to get the right feel.

Pattern 1: Configuration Bundle

Variables aren't just for tracking — they're **settings** you can tweak.

This bundle pattern is especially useful because:

- All tuning values are in **one place** (not scattered across blocks)
- You can **experiment** quickly by changing one number
- Other developers can understand your game's feel at a glance

Pattern 1: Configuration Bundle

We'll add speed, jumpForce, and gravity to our game.

Find a combination that feels fun to play with.

Pattern 2: Counter (Score)

The problem: How would you count how many coins a player collects?

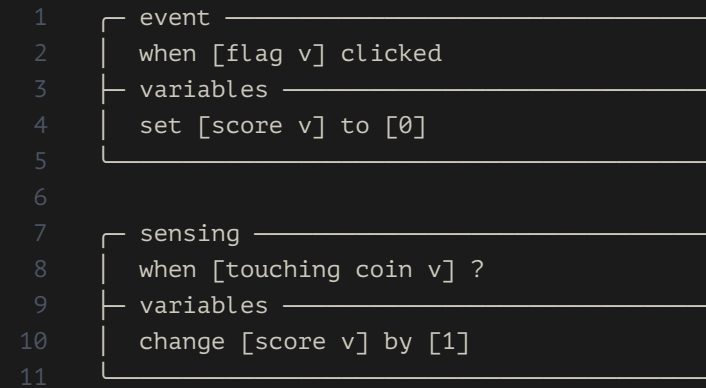
Every time the player touches a coin, the score goes up by 1.

Turn to a partner: describe in plain English what the program needs to do

We need:

1. A place to store the count → a **variable**
2. A starting value → **0**
3. Something that triggers the increase → **touching a coin**
4. The change itself → **+1**

Pattern 2: Counter (Score)



Two separate stacks:

- one to *initialize*,
- one to *respond* to an event.

Pattern 2: Counter (Score)

Step	Event	score	value
Start	Game starts		0
1	Touch coin 1		1
2	Touch coin 2		2
3	Touch coin 3		3
...	...		...

Each event → value goes up by exactly 1.

The variable **remembers** the total so far between events.

Pattern 2: Counter (Score)

We'll make a game where every time the player collects an item, the score increases by 1.

We'll also make different items worth different amounts.

Pattern 3: Countdown (Timer)

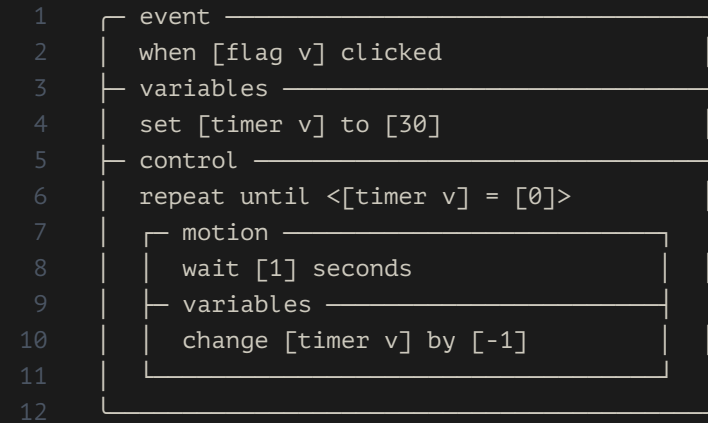
The problem: How do you make a timer that counts down from 30?

When the timer reaches 0, the game ends.

How is this different from counting up?

The change is **negative** instead of positive, and you **stop** when you reach a target value.

Pattern 3: Countdown (Timer)



negative change, loop, and stop condition.

Pattern 3: Countdown (Timer)

Step	Action	<code>timer</code> value
Start	Initialize	30
1	Wait 1 sec, change by -1	29
2	Wait 1 sec, change by -1	28
...	...	...
30	Wait 1 sec, change by -1	0 → stop

The variable **drives the loop** — the loop checks the value each time and stops when it hits 0.

Pattern 3: Countdown (Timer)

To the same game, we'll make it end after 15 seconds.

And then change how the player looks when there's only 5 seconds left.

Pattern 4: Flag (State Variable)

The problem: How does the game know whether it's playing, paused, or over?

You need a variable that stores the **current state** of the game.

Common states:

- "playing" ,
- "paused" ,
- "game over" ,
- "menu"

Pattern 4: Flag (State Variable)

```
1  ┌ event ───────────┐
2  │ when [flag v] clicked
3  └──────────────────┘
4  ┌ variables ───────┐
5  │ set [gameState v] to [playing]
6  └──────────────────┘
7  ┌ control ─────────┐
8  │ forever
9  │ ┌ control ───────┐
10 │ │ ... move sprites,
11 │ │   check collisions ...
12 │ │ └────────────────┘
13 │ │ if <[gameState v] = [game over]>
14 │ │ ┌────────────────┐
15 │ │ │ ... show game over screen ...
16 │ │ └────────────────┘
17 │ └────────────────┘
18 └──────────────────┘
```

Pattern 4: Flag (State Variable)

A flag variable doesn't count — it **switches** between values:

```
1  gameState:  menu  →  playing  →  game over
2              ↑           ↓
3              └─ paused ─┘
```

The variable acts as a **traffic light** — code checks it before deciding what to do.

This replaces a tangled mess of `if` blocks with a single variable that controls the flow.

Pattern 4: Flag (State Variable)

We'll add 2 states to our game:

- playing
- game over

And 2 states for the player:

- moving
- idle

Pattern 5: Max Tracker (High Score)

The problem: How do you remember the highest score ever achieved?

Every time the score changes, check if it's higher than the previous best.

What variable do we need beyond the current score?

A second variable: `highScore`. It only changes when `score` exceeds it.

Pattern 5: Max Tracker (High Score)

```
1  ┌ event ───────────┐
2  │ when [flag v] clicked
3  └──────────────────┘
4  ┌ variables ───────┐
5  │ set [highScore v] to [0]
6  └──────────────────┘
7
8  ┌ event ───────────┐
9  │ when [score changes]
10 └──────────────────┘
11 ┌ control ─────────┐
12 │ if <[score v] > [highScore v]>
13 │   ┌ variables ───┐
14 │   │ set [highScore v] to [score]
15 │   └──────────┘
16 └──────────────────┘
```

A **conditional update** — the variable only changes when a comparison is true.

Pattern 5: Max Tracker (High Score)

Round	Score after round	Comparison	highScore
1	15	$15 > 0 \rightarrow \text{true}$	15
2	12	$12 > 15 \rightarrow \text{false}$	15
3	22	$22 > 15 \rightarrow \text{true}$	22
4	18	$18 > 22 \rightarrow \text{false}$	22
5	30	$30 > 22 \rightarrow \text{true}$	30

`highScore` only moves **upward**. It holds the "best so far."

This pattern generalizes to: **compare** $\rightarrow$ **conditionally update** (works for min, max, closest, etc.)

Pattern 5: Max Tracker (High Score)

We'll add a high score to our game. If we click the flag, the score resets but the high score stays until we close the browser.

Summary: What all 5 patterns share

Each pattern is the same core idea:

A **named box** (variable) whose **contents change** over time.

The pattern is defined by *what* changes the value and *when*.

Pattern	What changes the value	When
Bundle	No change (it <i>drives</i>)	At start, then read only
Counter	Event	Each time event occurs
Countdown	Loop iteration	Each second until 0
Flag	Game event	When state transitions
Max Tracker	Another variable	When it exceeds current